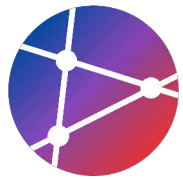


Shift Left Testing with Test Automation

테스트 자동화를 통한 품질 향상 전략과

GitHub Actions를 활용한 CI/CD 파이프라인 구축 실습



학습 목표

Learning Objectives



Shift Left Testing 개념 이해



테스트 피라미드 구조 파악



단위 테스트, 통합 테스트, E2E 테스트 작성



GitHub Actions에서 테스트 자동화 구현



TDD (Test-Driven Development) 실천 방법



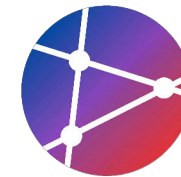
SECTION 01

Shift Left Testing

개발 초기 단계로 테스트를 이동하여
빠른 피드백과 품질 향상을 달성합니다.

Shift Left Testing

정의, 이점 및 비용 효율성 (Definition & Benefits)



정의 (Definition)

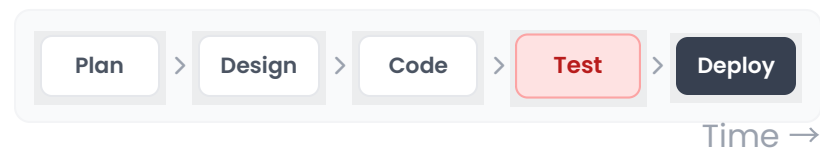
테스트 활동을 개발 프로세스의 **초기 단계(왼쪽)**로 이동시켜, 결함을 조기에 발견하고 품질을 확보하는 전략입니다.

KEY BENEFITS

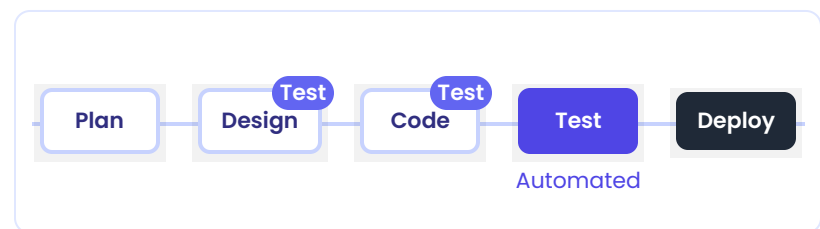
-  **버그 조기 발견**
Early Detection
-  **빠른 피드백**
Fast Feedback Loop
-  **안정적 배포**
Confident Deployment

프로세스 비교

TRADITIONAL (WATERFALL)



SHIFT LEFT MODEL



💡 각 단계마다 테스트 수행 (Continuous Testing)

버그 수정 비용

Relative Cost

발견 시점이 늦을수록 비용 급증

Dev. | \$100

Test | \$1k

Prod. | \$10k

비용 절감 효과

프로덕션 단계 수정은
개발 단계보다 **100배** 비쌉니다.



SECTION 02

Test Pyramid

빠르고 저렴한 단위 테스트를 기반으로,
통합과 E2E는 적정 비율을 유지합니다.



테스트 수준별 비교 분석

단위 vs 통합 vs E2E 테스트의 주요 특징 및 비용 차이

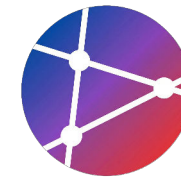


구분	주요 특징	실행 속도	비용/관리	신뢰성	권장 비율
 단위 테스트	개별 함수/클래스 격리 외부 의존성 없음, Mocking 활용	매우 빠름 (<1초) 	저렴 / 쉬움	중간	70%
 통합 테스트	컴포넌트 간 상호작용 DB, API 등 외부 시스템 연동 확인	중간 (1~10초) 	중간	높음	20%
 E2E 테스트	사용자 시나리오 전체 실제 브라우저 환경, UI/UX 검증	느림 (10초~분) 	비쌈 / 어려움	매우 높음	10%



Insight: 비용 vs 가치 트레이드오프

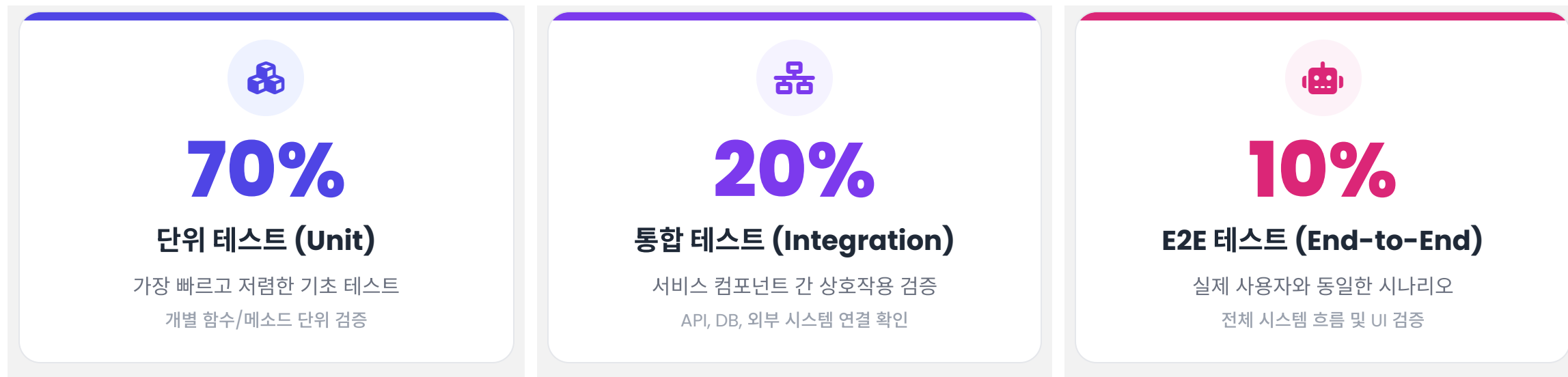
버그를 E2E 단계에서 발견하면 수정 비용이 높지만, 실제 사용자 경험에 가장 직접적인 영향을 줍니다. 반면, 단위 테스트는 저렴하게 대량으로 작성하여 코드 레벨의 논리적 오류를 빠르게 걸러내는 거름망 역할을 합니다. 따라서 피라미드 형태의 균형 잡힌 전략이 필수적입니다.



테스트 비율 권장사항

Recommended Test Ratios in Test Pyramid

이상적인 테스트 자동화 전략은 실행 속도가 빠르고 비용이 저렴한 단위 테스트를 기반으로 하며, 상위 레벨로 갈수록 테스트 수를 줄여 **유지 보수 비용을 최소화**합니다.



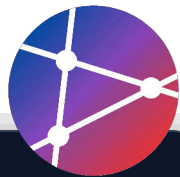


SECTION 03

Unit Tests

단위 테스트

Jest 및 컴포넌트 테스트, Mocking 기법을 통해
빠른 피드백을 확보하고 코드 품질을 높입니다.



Jest

기본 예제

1 구현 (Implementation)

테스트할 대상 함수(sum, multiply)를 작성하고 export합니다.

2 테스트 (Test)

describe 블록으로 그룹화하고, test 또는 it 함수로 케이스를 정의합니다.

3 검증 (Assertion)

expect(received).toBe(expected) 구문으로 결과를 검증합니다.

```
$ npm test
PASS ./sum.test.js
  ✓ adds 1 + 2 to equal 3 (2 ms)
  ✓ multiplies 2 * 3 to equal 6 (1 ms)

Test Suites: 1 passed, 1 total
Tests: 2 passed, 2 total
```

sum.js (구현)

```
export function sum(a, b) {
  return a + b;
}

export function multiply(a, b) {
  return a * b;
}
```

sum.test.js (테스트)

```
import { sum, multiply } from './sum';

describe('Math functions', () => {
  test('adds 1 + 2 to equal 3', () => {
    expect(sum(1, 2)).toBe(3);
  });

  test('multiplies 2 * 3 to equal 6', () => {
    expect(multiply(2, 3)).toBe(6);
  });
});
```

React 컴포넌트 테스트 (Jest)



컴포넌트 렌더링

render() 함수로 가상 DOM에 컴포넌트를 마운트하여 테스트 환경을 구성합니다.



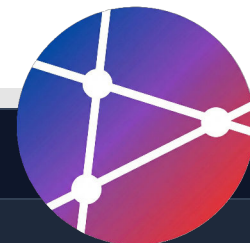
사용자 이벤트 시뮬레이션

fireEvent 등을 통해 클릭, 입력 등 사용자의 실제 상호작용을 재현합니다.



결과 검증 (Assertion)

expect()와 매치(toBeInTheDocument 등)로 UI 변화나 함수 호출을 확인합니다.



Button.test.jsx

```
// 1. 대상 컴포넌트 (Button.jsx)
export function Button({ onClick, children }) {
  return (
    <button onClick={onClick}>
      {children}
    </button>
  );
}

// 2. 테스트 코드 (Button.test.jsx)
import { render, screen, fireEvent } from '@testing-library/react';
import { Button } from './Button';

describe('Button component', () => {
  test('renders button with text', () => {
    // Arrange & Act: 렌더링
    render(<Button>Click me</Button>);

    // Assert: 텍스트 존재 확인
  });
});
```

단위 테스트

Mocking (fetch)



외부 의존성 격리

실제 API 서버를 호출하지 않고, 가짜(Mock) 함수를 사용하여 네트워크 의존성을 제거합니다.



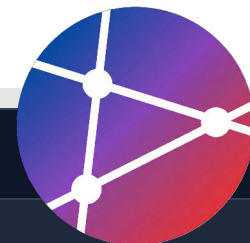
동작 제어

`mockResolvedValueOnce` 를 통해 테스트 시나리오에 맞는 응답 값을 강제로 반환합니다.



호출 검증

`toHaveBeenCalledWith` 로 함수가 올바른 인자와 함께 호출되었는지 확인합니다.



```
userService.test.js

// userService.js (대상 코드)
export async function fetchUser(id) {
  const response = await fetch(`/api/users/${id}`);
  return response.json();
}

// userService.test.js (테스트 코드)
import { fetchUser } from './userService';

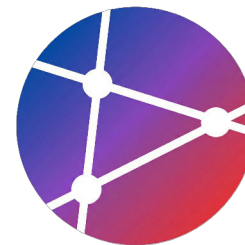
// 1. fetch 전역 함수 모킹
global.fetch = jest.fn();

describe('fetchUser', () => {
  beforeEach(() => {
    fetch.mockClear(); // 각 테스트 전 초기화
  });

  test('fetches user data', async () => {
    const mockUser = { id: 1, name: 'John' };

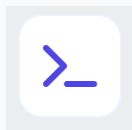
    // 2. 가짜 응답 설정 (Promise 반환 시뮬레이션)
    fetch.mockResolvedValueOnce(Promise.resolve({
      json: () => mockUser
    }));

    const user = await fetchUser(1);
    expect(user).toEqual(mockUser);
  });
});
```



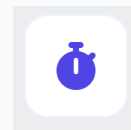
단위 테스트 실행과 팁

Best Practices for Running Unit Tests



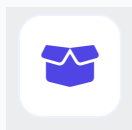
표준화된 실행 명령

`npm test` 명령어로 일관되게 실행하며, CI/CD 파이프라인의 첫 번째 관문(Gate)으로 설정하여 빠른 피드백을 확보합니다.



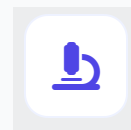
초고속 실행 속도

전체 단위 테스트 스위트는 수 초 내에 완료되어야 합니다. 개발자가 코드를 수정할 때마다 부담 없이 반복 실행할 수 있어야 합니다.



완벽한 격리 (Isolation)

데이터베이스, 네트워크, 파일 시스템 등 외부 의존성은 **Mocking**을 통해 제거하여 테스트의 독립성을 보장해야 합니다.



작은 테스트 범위

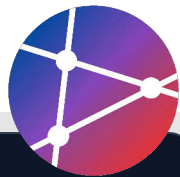
하나의 테스트 케이스는 하나의 개념(함수, 클래스, 메소드)만 검증하며, 실패 시 원인을 즉시 파악할 수 있도록 작게 유지합니다.



SECTION 04

Integration Testing

API, 데이터베이스 등 여러 컴포넌트 간의 상호작용을 검증하여 시스템 신뢰성을 확보합니다.



INTEGRATION TEST

API 엔드포인트 테스트 (Express)



Supertest 활용

서버 구동 없이 Express 앱의 HTTP 요청/응답을 시뮬레이션합니다.



상태 코드 & 데이터 검증

HTTP Status(200, 404)와 JSON 응답 구조를 동시에 검증합니다.



비동기 테스트

async/await로 비동기 API 요청을 직관적으로 처리합니다.

```
app.test.js

// 1. 필요한 모듈 импорт
import request from 'supertest';
import app from './app'; // Express 앱 인스턴스

describe('GET /api/users/:id', () => {

  // ✅ 성공 케이스: 사용자 데이터 반환
  test('returns user data for valid id', async () => {
    // app에 GET 요청 전송 및 200 OK 기대
    const response = await request(app)
      .get('/api/users/1')
      .expect(200);

    // 응답 바디 데이터 검증
    expect(response.body).toHaveProperty('id', 1);
    expect(response.body).toHaveProperty('name');
  });

  // ❌ 실패 케이스: 존재하지 않는 사용자
  test('returns 404 for non-existent user', async () => {
```



데이터베이스 통합 테스트



실제 DB 상호작용

Mock 대신 실제 DB(또는 인메모리 DB)를 사용하여 쿼리와 스키마 정합성을 검증합니다.



테스트 격리 (Isolation)

beforeEach에서 데이터를 초기화하여 각 테스트가 서로 영향을 주지 않도록 합니다.



연결 생명주기 관리

beforeAll로 연결하고 afterAll로 해제하여 리소스 누수를 방지합니다.

userRepository.test.js

```
import { UserRepository } from '../userRepository';
import { setupTestDB, teardownTestDB } from '../testUtils';

describe('UserRepository Integration', () => {
  let repo;

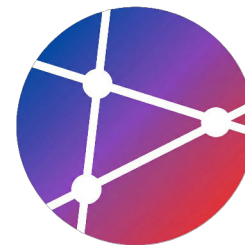
  // 테스트 스위트 시작 전 DB 연결
  beforeAll(async () => {
    await setupTestDB();
    repo = new UserRepository();
  });

  // 테스트 스위트 종료 후 연결 해제
  afterAll(async () => {
    await teardownTestDB();
  });

  // 각 테스트 실행 전 데이터 초기화 (격리 보장)
  beforeEach(async () => {
    await db.users.deleteMany({});
  });
```


통합 테스트 환경 설정

Integration Test Environment Setup



DB 초기화 및 정리 전략

✓ 테스트 격리 (Isolation)

각 테스트 실행 전/후에 상태를 초기화하여 다른 테스트에 영향을 주지 않도록 보장

↺ 트랜잭션 롤백 (Transaction Rollback)

가장 빠른 방법. 테스트 종료 시 커밋하지 않고 롤백 수행

✂ Truncate / Clean

롤백이 불가능한 경우 모든 테이블의 데이터를 삭제 (예: NoSQL)



외부 의존성 관리 기준



Testcontainers (권장)

신뢰성 높음

실제 DB/서비스를 Docker 컨테이너로 띄워 테스트. 운영 환경과 가장 유사함



In-memory DB

매우 빠름

SQLite/H2 사용. 설정이 쉽고 빠르지만, 특정 쿼리 문법 차이로 인한 버그 발생 가능성 있음



Mocking

제어 불가능한 3rd Party API(결제, 이메일)는 모킹하여 호출 여부만 검증



SECTION 05

End-to-End Testing

사용자 시나리오를 기반으로
시스템의 전체 흐름을 완벽하게 검증합니다.



E2E 테스트

Playwright 예제



Page Fixture

`page` 객체 하나로 브라우저 컨텍스트 격리 및 제어 자동화



User Actions

`fill`, `click` 등 직관적인 API로 사용자 동작 시뮬레이션



Web-First Assertions

조건이 충족될 때까지 자동으로 재시도(Auto-waiting)하는 강력한 단언문



디버깅 용이성

헤드리스 모드 및 트레이스 뷰어(Trace Viewer) 지원

```
e2e/login.spec.js

import { test, expect } from '@playwright/test';

test.describe('Login flow', () => {
  // 1. 성공적인 로그인 케이스
  test('successful login', async ({ page }) => {
    await page.goto('http://localhost:3000/login');

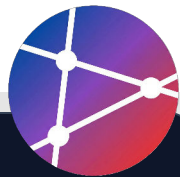
    // 폼 입력
    await page.fill('input[name="email"]', 'user@example.com');
    await page.fill('input[name="password"]', 'password123');

    // 로그인 버튼 클릭 및 네비게이션 대기
    await page.click('button[type="submit"]');

    // URL 및 UI 요소 검증 (자동 대기 포함)
    await expect(page).toHaveURL('http://localhost:3000/dashboard');
    await expect(page.locator('h1')).toContainText('Dashboard');
  });

  // 2. 실패 케이스 (잘못된 자격 증명)
  test('failed login with invalid credentials', async ({ page }) => {
    await page.goto('http://localhost:3000/login');

    await page.fill('input[name="email"]', 'wrong@example.com');
```



Cypress 예제

체크아웃 시나리오



User Flow 검증

제품 검색부터 주문 완료까지 전체 프로세스 자동화



직관적인 문법

jQuery 스타일의 체이닝(cy.get)으로 높은 가독성 제공



자동 대기 (Wait)

요소가 상호작용 가능해질 때까지 Cypress가 자동 대기

```
cypress/e2e/checkout.cy.js

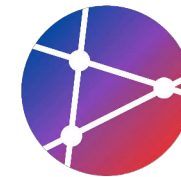
describe('Checkout flow', () => {
  beforeEach(() => {
    cy.visit('http://localhost:3000');
  });

  it('completes a purchase', () => {
    // 1. 제품 검색 및 선택
    cy.get('input[placeholder="Search"]').type('laptop');
    cy.get('button[type="submit"]').click();

    cy.contains('MacBook Pro').click();
    cy.contains('Add to Cart').click();

    // 2. 장바구니 확인
    cy.get('.cart-icon').click();
    cy.contains('MacBook Pro').should('be.visible');
    cy.contains('Checkout').click();

    // 3. 배송 및 결제 정보 입력
    cy.get('input[name="address"]').type('123 Main St');
    cy.get('input[name="cardNumber"]').type('4111...');
    cy.get('input[name="cvv"]').type('123');
```



E2E 테스트 시나리오 설계

Designing Reliable & Maintainable E2E Tests



핵심 사용자 여정 선정

비즈니스 가치가 높은 흐름에 집중

- 모든 기능을 E2E로 테스트하지 않음 (비용 최적화)
- Happy Path와 치명적 에러 케이스 위주 선정
- 예: 회원가입 → 로그인 → 상품 검색 → 결제 완료



안정적인 셀렉터 사용

변경에 강한(Resilient) 테스트 코드

- CSS 클래스, 태그 구조 대신 사용자 관점 속성 사용
- `data-testid="submit-btn"` 등 명시적 속성 활용
- `getByText`, `getByRole` 우선 사용 권장



비동기 대기 처리

Flaky Test 방지를 위한 대기 전략

- 고정 시간 대기(`sleep(1000)`) 사용 금지
- 요소의 가시성(Visibility)이나 API 응답 대기 사용
- Playwright/Cypress의 내장 Auto-waiting 적극 활용



디버깅 정보 확보

실패 시 신속한 원인 파악 환경

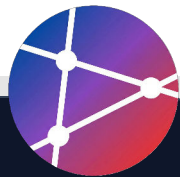
- 테스트 실패 시 스크린샷/비디오 자동 캡처 설정
- CI 환경(GitHub Actions) Artifact로 결과 저장
- Trace Viewer를 통한 실행 흐름 단계별 역추적



SECTION 06

GitHub Actions Automation

CI/CD 파이프라인을 구축하여
테스트 실행과 커버리지 측정을 자동화합니다.



GitHub Actions

기본 CI 워크플로우



Trigger Events

Main 브랜치에 대한 push 및 pull_request 이벤트 발생 시 자동으로 워크플로우가 실행됩니다.



Environment Setup

ubuntu-latest 실행기에서 Node.js 환경(v20)을 구성하고 npm 의존성을 캐싱하여 설치 속도를 높입니다.



Test Pipeline

의존성 설치(ci) 후, 린트(Lint) → 단위 테스트 → 통합 테스트 순서로 품질 검증을 수행합니다.

```
.github/workflows/test.yml

name: Test

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      # 1. 코드 체크아웃
      - uses: actions/checkout@v4

      # 2. Node.js 환경 설정 & 캐싱
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      # 3. 의존성 설치 (Clean Install)
```



멀티 환경 테스트

Matrix Strategy



Matrix 전략

단일 설정으로 OS 및 Node 버전의 모든 조합을 자동 생성하여 테스트합니다.



크로스 플랫폼 검증

Ubuntu, Windows, macOS 등 다양한 환경에서의 호환성을 보장합니다.



병렬 실행 효율성

3개 OS × 3개 버전 = 총 9개의 작업이 동시에 실행되어 피드백이 빠릅니다.

```
.github/workflows/test-matrix.yml

name: Test Matrix
on: [push, pull_request]

jobs:
  test:
    runs-on: ${{ matrix.os }}

    strategy:

      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
        node-version: [18, 20, 21]

    steps:
      # 코드 체크아웃
      - uses: actions/checkout@v4

      # Node.js 설정 (매트릭스 변수 사용)
      - name: Setup Node.js ${{ matrix.node-version }}
```


E2E 자동화 설정

Playwright Workflow



브라우저 설치

CI 환경에는 브라우저가 없으므로 playwright install로 필요한 바이너리를 설치해야 합니다.



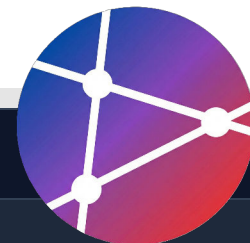
앱 실행 및 대기

테스트 전 npm start &로 백그라운드 실행 후, wait-on으로 서버 준비를 기다립니다.



결과 업로드

upload-artifact를 사용하여 실패 스크린샷이나 테스트 리포트를 저장합니다.



.github/workflows/e2e.yml

```
name: E2E Tests

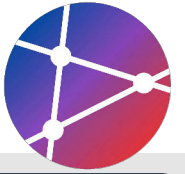
on: [push, pull_request]

jobs:
  e2e:
    runs-on: ubuntu-latest

    steps:
      # 1. 소스 코드 체크아웃
      - uses: actions/checkout@v4

      # 2. Node.js 설정
      - uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install dependencies
        run: npm ci
```



테스트 커버리지 자동화 (Coverage)



커버리지 리포트

`jest --coverage` 옵션으로 실행 결과를 생성하고 Codecov 등 외부 서비스로 업로드합니다.



품질 게이트 (Threshold)

커버리지가 특정 기준(예: 80%) 미만일 경우 CI 파이프라인을 실패시켜 품질을 강제합니다.



가시성 확보

PR 단계에서 커버리지 증감 여부를 시각적으로 확인하여 코드 리뷰 효율을 높입니다.

.github/workflows/coverage.yml

```
name: Coverage

on: [push, pull_request]

jobs:
  coverage:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install dependencies
        run: npm ci
```

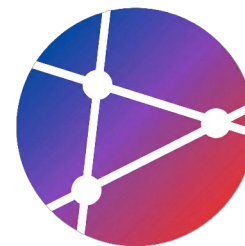


SECTION 07

Test-Driven Development

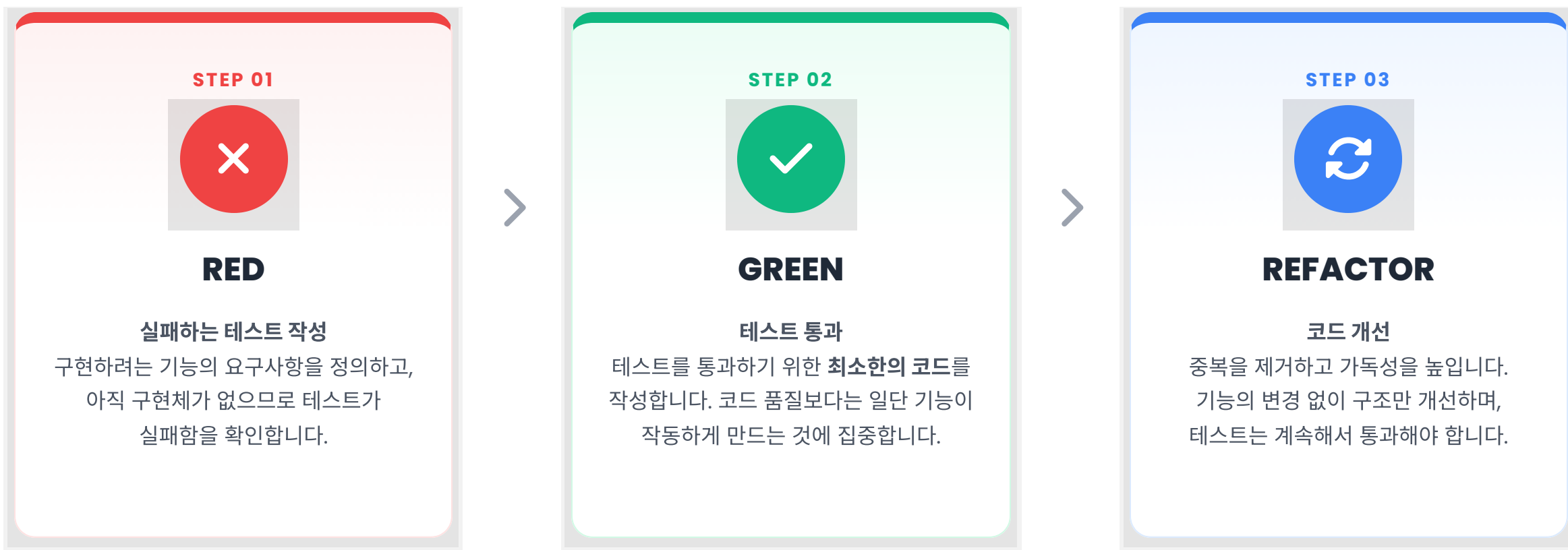
Red-Green-Refactor 사이클을 통해
설계 개선과 코드 품질을 동시에 달성합니다.





Red-Green-Refactor 사이클

TDD (Test-Driven Development)의 핵심 프로세스





STEP 1 Test-Driven Development

TDD 실습: RED 실패하는 테스트 작성



테스트 우선 작성

구현하려는 기능(TodoList)이 아직 존재하지 않지만, 기대하는 동작을 코드로 먼저 정의합니다.



의도적인 실패 확인

테스트를 실행하여 실패(Red)하는지 확인합니다. 이는 테스트가 올바르게 동작하고 있음을 증명합니다.



요구사항 명세

"할 일을 추가하고 목록을 조회한다"는 요구사항이 테스트 코드로 명확히 문서화됩니다.

```
todo.test.js

// 1. 아직 구현되지 않은 TodoList 클래스를 import
import { TodoList } from './todo';

describe('TodoList', () => {
  test('adds a new todo', () => {
    // Arrange: TodoList 인스턴스 생성 (실패 예상)
    const list = new TodoList();

    // Act: 할 일 추가
    list.add('Buy milk');

    // Assert: 목록 조회 및 검증
    expect(list.getAll()).toEqual([
      { id: 1, text: 'Buy milk', completed: false }
    ]);
  });
});

FAIL ./todo.test.js
• TodoList > adds a new todo
```

TDD 실습 예제 (2)

GREEN & REFACTOR



Step 2: GREEN

테스트를 통과하기 위한 **최소한의 코드**를 작성합니다. 완벽한 설계보다 통과가 우선입니다.



Step 3: REFACTOR

중복을 제거하고 가독성을 개선합니다. 테스트가 보호막 역할을 하므로 안전하게 수정할 수 있습니다.

🔄 이후 다음 기능(예: complete) 테스트 작성으로 이동

todo.js

// Step 2: GREEN - 테스트 통과를 위한 최소 구현

```
export class TodoList {
  constructor() {
    this.todos = [];
    this.nextId = 1;
  }

  add(text) {
    this.todos.push({
      id: this.nextId++,
      text: text,
      completed: false,
    });
  }

  getAll() {
    return this.todos;
  }
}
```





SECTION 08

Test Quality Metrics

메트릭과 테스트 강건성으로
소프트웨어 신뢰도를 향상시킵니다.



코드 커버리지 메트릭 및 설정



Jest 설정 (package.json)

`coverageThreshold`를 사용하여 전역 품질 기준(예: 80%)을 강제합니다.



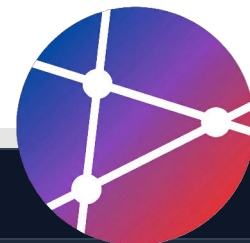
실행 명령

`npm run test:coverage` 명령으로 테스트 실행과 동시에 리포트를 생성합니다.



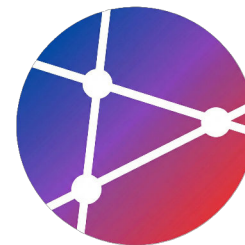
주요 분석 지표

구문(Stmts), 분기(Branch), 함수(Funcs), 라인(Lines) 커버리지를 확인합니다.



```
package.json & Terminal Output

// 1. package.json 설정
{
  "scripts": {
    "test": "jest",
    "test:coverage": "jest --coverage"
  },
  "jest": {
    "coverageThreshold": {
      "global": {
        "branches": 80,
        "functions": 80,
        "lines": 80,
        "statements": 80
      }
    }
  }
}
```

테스트 Best Practices

효과적이고 유지보수 가능한 테스트 작성을 위한 4가지 원칙

1
2

AAA 패턴 (Arrange-Act-Assert)

테스트 코드를 준비(Arrange), 실행(Act), 검증(Assert) 단계로 명확히 분리하여 가독성을 높입니다.



테스트 독립성 (Independence)

각 테스트는 서로 의존하지 않아야 하며, 실행 순서에 관계없이 항상 동일한 결과를 보장해야 합니다.



명확한 이름 (Clear Naming)

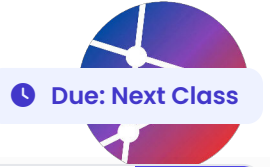
'test1' 같은 모호한 이름 대신, 테스트의 시나리오와 예상 결과가 드러나는 구체적인 문장형 이름을 사용합니다.



하나의 개념만 테스트

한 테스트 함수에서는 하나의 개념(기능)만 검증하여, 실패 시 원인을 즉시 파악할 수 있도록 합니다.

실습 과제 (Assignments)



단위 테스트 작성

Task 01

- ✓ 간단한 함수(계산기, 문자열 처리 등) 로직 구현
- ✓ Jest 프레임워크를 사용하여 테스트 케이스 작성
- ✓ 테스트 커버리지 **80% 이상** 달성 목표
- ✓ GitHub Actions 연동하여 CI 파이프라인 구성



TDD 실천 (Todo App)

Task 02

- ✓ **Red-Green-Refactor** 개발 사이클 준수
- ✓ 테스트 코드 선행 작성 후 기능 구현 (Test-First)
- ✓ 5개 이상의 주요 CRUD 기능 구현 완료
- ✓ Git 커밋 메시지에 TDD 단계 명시 (feat/test/refactor)



E2E 테스트 자동화

Task 03

- ✓ Playwright 또는 Cypress 프로젝트 설정
- ✓ 핵심 사용자 시나리오 3개 이상 테스트 코드 작성
- ✓ GitHub Actions 워크플로우에 E2E 테스트 추가
- ✓ 테스트 실패 시 스크린샷/비디오 아티팩트 저장 설정



레거시 코드 테스트

Task 04

- ✓ 테스트가 없는 기존 레거시 코드 분석
- ✓ 안전한 리팩토링을 위한 캐릭터라이제이션 테스트 추가
- ✓ 리팩토링 전후 테스트 통과 확인 및 커버리지 개선 리포트



제출 형식

GitHub Repository 링크 (README.md에 실행/리포트 포함)



평가 포인트

커버리지 달성도, TDD 사이클 준수 여부, CI 동작 확인

핵심 요약

Key Takeaways from Week 12



Shift Left

테스트를 개발 초기 단계로 이동하여 버그 수정 비용을 줄이고 품질을 조기에 확보합니다.



테스트 피라미드

단위(70%), 통합(20%), E2E(10%)의 균형 잡힌 비율로 신뢰성 높은 테스트를 구축합니다.



TDD 실천

Red-Green-Refactor 사이클을 통해 구현 전에 설계를 검증하고 견고한 코드를 작성합니다.



CI 자동화

GitHub Actions를 활용하여 모든 커밋에 대해 자동으로 테스트를 실행하고 모니터링합니다.



"Code without tests is broken by design."

— Jacob Kaplan-Moss

Django Co-creator

NEXT WEEK

Advanced Testing & Refactoring

- ✓ 테스트 품질 향상을 위한 고급 기법
- ✓ 코드 스멜(Code Smell) 식별
- ✓ 안전한 리팩토링 전략